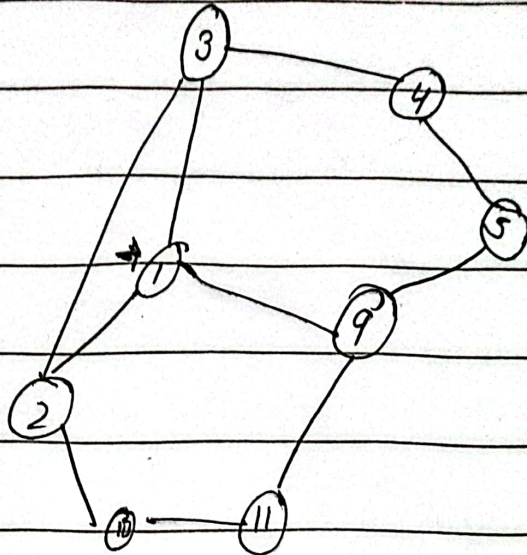


Graph traversal:

→ BFS (Breadth first search) ⇒ (level by level)

2/4/21



BFS:

→ we can start traversing from any point (until not mention in Ques)

→ We can go anywhere.

→ We use que in BFS / visited que

Solution

Starting from ①

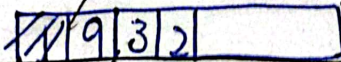
Date: _____



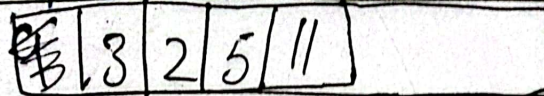
Output:



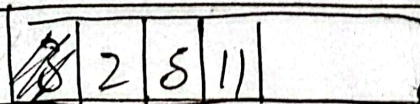
~~1, 9, 3, 2~~



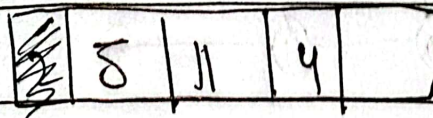
1



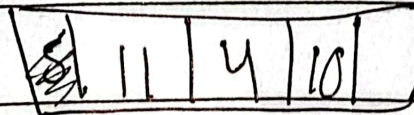
: 1, 9



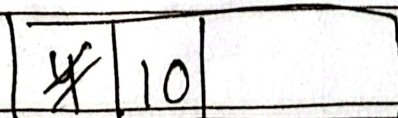
1, 9, 3



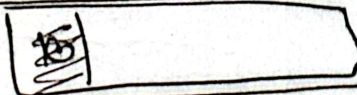
1, 9, 3, 2



1, 9, 3, 2, 5



1, 9, 3, 2, 5, 11



1, 9, 3, 2, 5, 11, 4

1, 9, 3, 2, 5, 11, 4

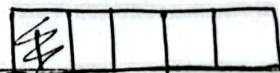
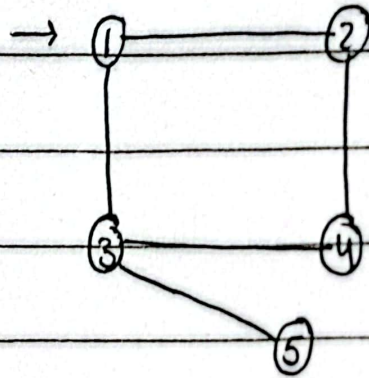
1, 9, 3, 2, 5, 11, 4

$\{1, 2, 3, 4, 5\}$

$\{(1, 2), (1, 3), (2, 4), (3, 4), (3, 5)\}$

a) BFS

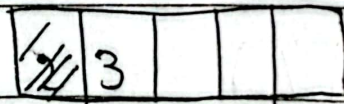
Start from 1:



① is connected with

Output:

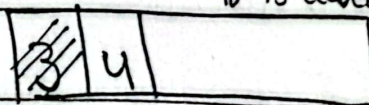
1



② is connected with

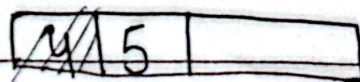
we will not write ① because it is already visited.

1, 2



③ is connected with

1, 2, 3



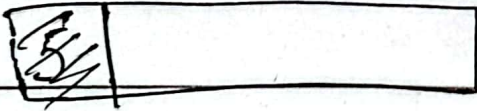
Day: _____

Date: _____

~~Agarwal~~

we will not write 3, 1

because it is already
visited



As 5 is ~~connected~~ connected

with 3 and 3 is already
visited so

we will not write
is

output :

1, 2, 3, 4,

1, 2, 3, 4, 5

Final output.

Day: _____

Date: _____

Agar Agar

(DFS)

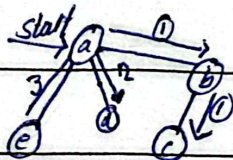
Depth-First Search.

→ we can start traversing from any point.

(until no mentioned)

→ we traverse all adjacent vertices one by one.

Agar Agar



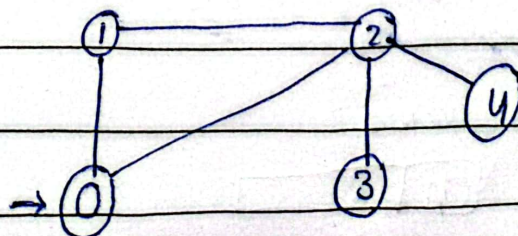
from given first way, we go from (a) towards (b) then (c) until we reach ~~each each~~ ^{end.} and then come back to start point if there are more than one way.

second way is (a) to (d)

third way is (a) to (e).

→ we use stack for backward movement.

Example:



Day: _____

Date: _____

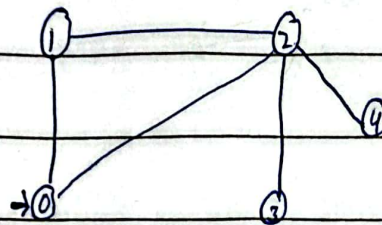
Output: 0 1 2 3 4

Explanation:

The source vertex is 0

start at 0: we marked it as visited

output: [0] [] [] [] []



stack



we have two ways

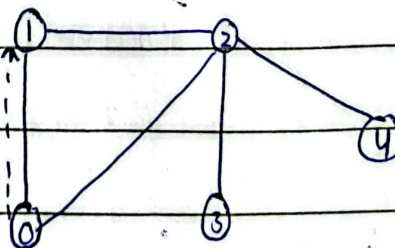
(0) to (1)

or

(0) to (2)

for all step
we using this
stack

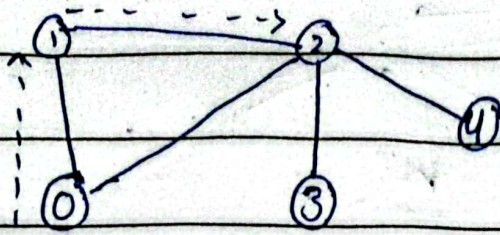
Move to 1:



Mark ① visited

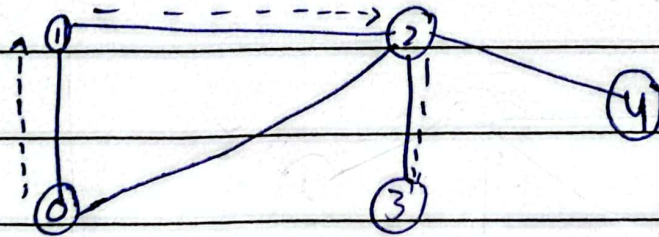
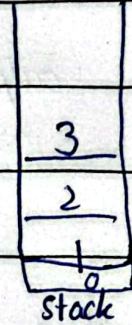
output: [0] [1] [] [] []

Move to 2:



output: [0][1][2][][][][]

Move to 3:



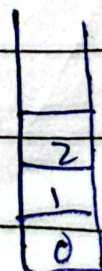
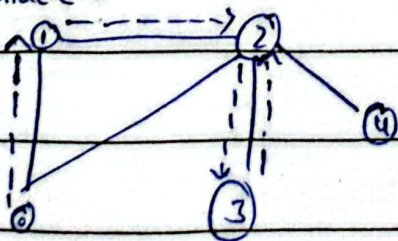
output: [0][1][2][3][][]

∴ now back track from 3 to 2

we use stack for this movement

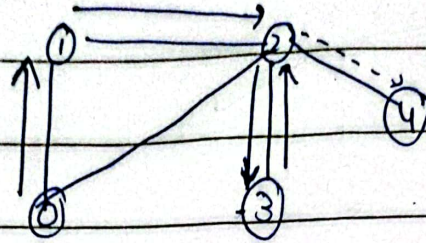
because last in first out we have
the point where to go back
some remove 3 from stack

'Backtrack:3



updated
stack.

move to 4.

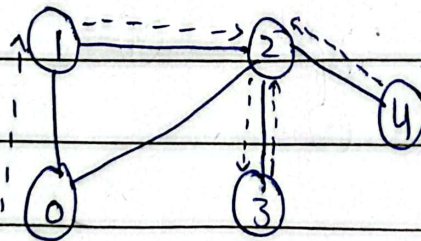


4
2
1
0

output :

0	1	2	3	4
---	---	---	---	---

back track from 4

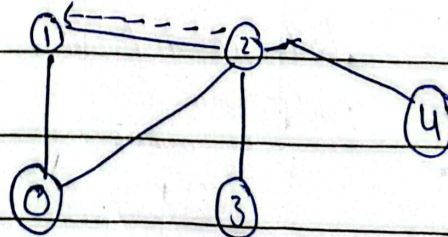


2
1
0

output will be same

0	1	2	3	4
---	---	---	---	---

back tracking from 2



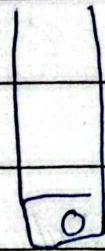
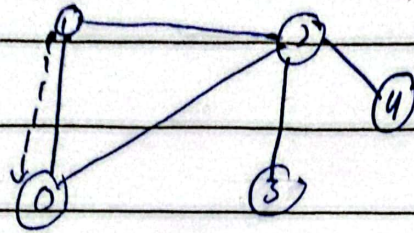
1
0

Day: _____

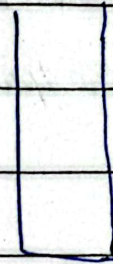
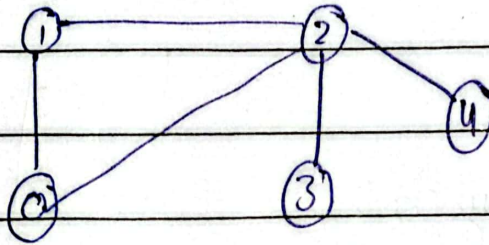
Ayan Ahmed

Date: _____

backtracking from 1



Back to source 0:



stack is empty.

Conclusion / justification of step:

start at 0: ^{mark as} visited, output: 0

Move to 1: " , output: 1

Move to 2: " , output: 2

Move to 3: " , output: 3 , back-track to 2

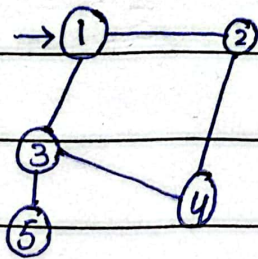
Move to 4: " , output: 4 , back-track to 2 then 1 then 0

Assignment Question: (DFS)

vertices : $\{1, 2, 3, 4, 5\}$

edges : $\{(1, 2), (1, 3), (2, 4), (3, 4), (3, 5)\}$

Graph:



Here we have ending point
5 and 4, from where
we are not going anywhere.

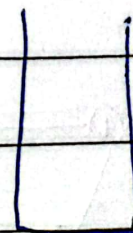
Solution

visited[] :

1	2	3	4	5

output[] :

--	--	--	--	--



stack
(empty).

Day: _____

Start from 1:

visited:

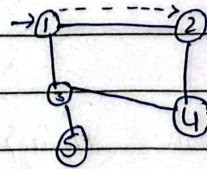
1	2	3	4	5
T	F	F	F	F

output:

1				
---	--	--	--	--

1
↓
stack

move to 2



visited:

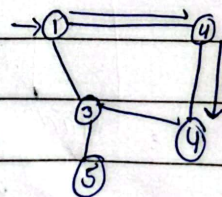
1	2	3	4	5
T	T	F	F	F

output:

1	2			
---	---	--	--	--

2
↓
stack

move to 4:



Agarwal

Date: _____

visited:

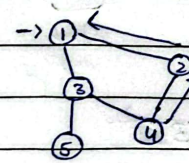
1	2	3	4	5
T	T	F	T	F

output:

1	2	4		
---	---	---	--	--

4
2
1

backtrack from 4 to 1



1

move to 3:

visited:

1	2	3	4	5
T	T	T	T	F

output:

1	2	4	3	
---	---	---	---	--

3
1

now we will not move to 4 because it is already visited.

move to 5

visited:

1	2	3	4	5
T	T	T	T	T

output:

1	2	4	3	5
---	---	---	---	---

5
3
1

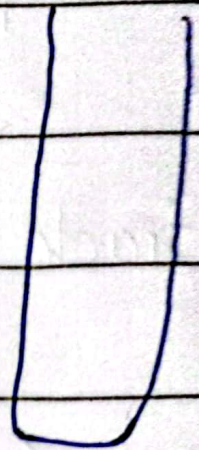
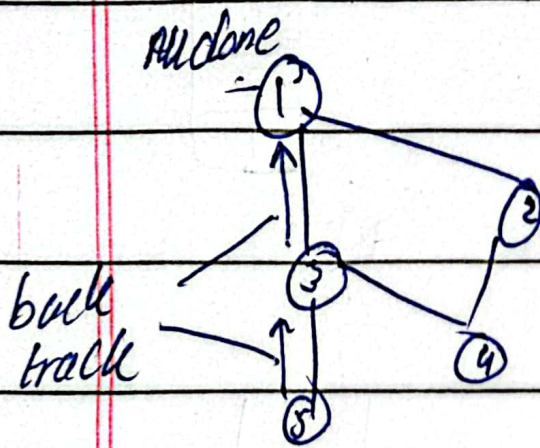
Day: _____

Answer

D₂

Answer

•) back tracking from 5 to 1



stack
empty

output :

1	2	4	3	5
---	---	---	---	---

Ans